AGENTOPS INTELLIGENCE

Complete Project Blueprint

High-Level Design (HLD) • Low-Level Design (LLD)
Architecture • APIs • Data Model • Evaluation • Deployment

A research-oriented, open-source platform for evaluating tool-using AI agents under controlled failures and risk constraints.

Target: AI/ML Engineer portfolio + reproducible research
Deployment goal: ■0 using free-tier services and local reproducibility

Version 1.0 • Blueprint and proposed design, not a claim of completed implementation or validated results.

Document Guide

This document is intended to guide implementation from first prototype through reproducible experiments and a public demonstration. It distinguishes planned features and research hypotheses from results that must be measured.

Section	What it covers
1. Executive summary	What the project is, who benefits, and why it is useful
2. Scope and requirements	Goals, non-goals, personas, functional and quality requirements
3. Research plan	Research question, hypotheses, novelty, baseline and ablation strategy
4. HLD	System context, component architecture, deployment view, data flow
5. LLD	Modules, contracts, state machine, policy and verification logic
6. Data and persistence	Entities, schema, event model, retention and reproducibility
7. API specification	Endpoints, request/response examples, error model
8. Security and reliability	Threat model, guardrails, isolation and operational limits
9. Evaluation protocol	Metrics, benchmark design, statistics, experiment matrix
10. Repository and implementation	Target file structure and six-week implementation plan
11. Free-tier deployment	Recommended deployment approach and failure modes
12. Publication and portfolio	Paper structure, artifacts, demo and resume positioning

Key principle

Build a deterministic benchmark and policy/verification core before adding sophisticated planning or multiple agents. The research contribution must come from a testable method and reproducible evidence—not from the use of fashionable frameworks.

1. Executive Summary

AgentOps Intelligence is a proposed open-source platform for executing, observing, and evaluating AI agents that use tools to complete bounded tasks. It combines a workflow runtime, tool permission policies, evidence-based verification, bounded recovery, trace collection, and a reproducible benchmark.

The project is deliberately framed as an **agent reliability and evaluation system**, rather than another general-purpose chatbot. A user gives the system a goal and a controlled tool environment. The runtime plans and performs allowed operations, records each step, checks results against deterministic rules where possible, and stops, retries, or asks for approval when required.

1.1 The problem

Tool-using agents can fail in ways that a fluent final response conceals: they may call the wrong tool, misread tool output, repeat failed operations, exceed budgets, act without required permission, or report completion without satisfying the task. Measuring only the final answer misses important operational risks.

1.2 The proposed solution

- **Bounded execution:** maximum steps, tool calls, retries, runtime and optional model budget.
- **Risk-aware policy:** allow, deny or require approval for actions based on tool and argument risk.
- **Verification:** deterministic assertions, evidence references and explicit success criteria.
- **Controlled recovery:** retry transient failures, replan within limits, or escalate to human review.
- **Observability:** ordered events, tool traces, policy decisions, errors and latency.
- **Benchmarking:** repeatable tasks, injected failures, baseline comparisons and exportable results.

1.3 Who benefits and how

Audience	Usefulness
AI/ML engineers	Demonstrates agent runtime design, tool integration, testing, safety controls and evaluation.
AI platform teams	Provides a small testbed for comparing agent configurations and tracing failures.
Researchers	Provides task definitions, ground truth, controlled fault scenarios, ablations and reproducible artifacts.
Hiring teams	Can inspect code, run a demo, review tests, and evaluate measured results instead of relying on claims.
Developers building agents	Can test failure recovery and policy behavior before connecting consequential tools.

1.4 Success means

- A new developer can install the project and run the deterministic tests and benchmark using documented commands.
- Every run has a trace and a defined terminal status; failures are not silently dropped.
- Policy enforcement is implemented outside the LLM and is covered by tests.
- Results compare explicit baselines and ablations with uncertainty and limitations.
- A small public demo uses synthetic data, restrictive limits and no requirement for paid inference.

2. Scope, Requirements and Assumptions

2.1 Goals

- Evaluate how verification, risk-aware permissions and bounded recovery affect task success, policy violations, undetected failures, cost and latency.
- Support deterministic mock execution so core behavior is testable without API keys.
- Support pluggable model adapters without tying the benchmark to one provider.
- Expose tasks, runs, approvals, traces and experiment results through a versioned API.
- Publish reproducible task data, configuration files, test fixtures and sanitized results.

2.2 Non-goals for the first release

- No unrestricted autonomous operation on real business systems.
- No arbitrary shell execution, unrestricted SQL, or direct production writes.
- No claim of universal agent reliability or guaranteed correctness.
- No requirement for a multi-agent architecture; begin with one bounded agent and modular services.
- No mandatory vector database. Add Qdrant only if semantic retrieval is needed for a benchmark domain.
- No dependence on always-on free infrastructure for experiments or research reproducibility.

2.3 User roles

Role	Permissions and responsibilities
Researcher / owner	Defines tasks, policies, experiment configurations, and scoring rules; exports results.
Demo user	Runs approved sample tasks and views sanitized traces and reports.
Reviewer / approver	Approves or rejects a specific proposed action; decisions are audited.
System operator	Configures deployment, quotas, secrets, retention and service health.

2.4 Quality requirements

Requirement	Design target
Reproducibility	Persist configuration, dataset version/hash, tool versions, seed and model identifier.
Safety	Enforce allowlists, input validation, hard limits and approval checks outside the LLM.
Reliability	Explicit state transitions, bounded retries, idempotency where applicable, and failure records.
Observability	Ordered event log, timestamps, tool outcomes, policy results and timing metrics.
Portability	Local-first core, locked dependencies, Docker support, SQLite default.
Cost control	Deterministic default; optional model adapters and explicit usage limits.
Privacy	Synthetic benchmark data by default; redact secrets and avoid unnecessary personal data.

2.5 Assumptions

This blueprint describes intended design. Exact framework versions, hosting quotas, model access, and free-tier terms must be checked when implementation begins. The six-week plan is a focused target, not a guaranteed schedule. No performance results are claimed here.

3. Research Plan and Potential Contribution

3.1 Working research title

Risk-Aware Verification and Bounded Recovery for Reliable Tool-Using AI Agents

Proposed artifact: **AgentOps-Bench**, a reproducible benchmark for measuring tool-using agent behavior under controlled faults and risk constraints.

3.2 Research question

How do deterministic verification, risk-aware tool permissions, and bounded recovery affect task success, safety and execution cost when agents operate in environments with controlled tool failures?

3.3 Hypotheses to test

Hypothesis	Comparison	Primary outcomes
H1	Baseline vs. verification enabled	Undetected failure rate; task success; added latency
H2	Uniform vs. risk-aware permissions	Policy violation rate; safe completion rate; false denials
H3	No recovery vs. bounded recovery	Recovery success; tool calls; latency; cost
H4 (optional)	Individual components vs. full system	Interactions, trade-offs and combined effects

3.4 Potential novelty

- A clearly specified combination of risk-aware tool policy, deterministic outcome verification and bounded recovery.
- Fault-injection scenarios with known ground truth and a reproducible execution protocol.
- Joint reporting of task success, unsafe actions, undetected failures, recovery, latency and cost.
- A controlled ablation study showing which component helps, hurts or adds overhead.

Novelty warning

These are proposed contributions, not verified claims of novelty. Conduct a literature review of agent benchmarks, tool-use security, runtime verification, recovery methods and agent observability before asserting that the combination is new. Existing work may overlap substantially.

3.5 Baselines and experimental controls

- Baseline: same task, tools and limits; no added verification, risk-aware policy or recovery.
- Use matched initial environment states and identical task versions across variants.
- Begin with deterministic mock models to validate instrumentation and policy behavior.
- Use real model adapters only where a permitted free endpoint is available; record model/version and parameters.
- Predefine success conditions and failure categories before comparing variants.
- Run all eight combinations of the three binary components for a full 2×2×2 factorial study when practical.

3.6 Threats to validity

- Synthetic tasks may not represent production workflows.
- A small benchmark may have low statistical power or overrepresent hand-designed cases.
- LLM outputs can vary between runs even with a fixed seed.
- Model-based judges can share biases with the model being evaluated.
- Fault injection may favor the recovery policy if scenarios are too predictable.

- A risk policy may improve safety by refusing too much; report task completion and false denials together.

4. High-Level Design (HLD)

4.1 System context

The platform accepts a task and an experiment configuration, executes the agent against a restricted tool environment, records all significant events, verifies outcomes and exposes results through an API and dashboard.

Researcher / demo user
Web UI or CLI → FastAPI REST API
Task validator and run manager
Agent orchestrator / state machine
Policy engine ↔ Verification engine
Tool registry: calculator · read-only SQL · dataset search · simulated actions · fault injector
Event recorder → persistence → evaluator → reports

4.2 Component responsibilities

Component	Responsibility	Key boundary
API layer	Request validation, auth, rate limits, response schemas	Does not execute model-selected tools directly
Run manager	Creates run IDs, loads config, enforces global limits	Owns run lifecycle and cancellation
Orchestrator	Plans next step, observes results, advances state	Cannot bypass policy or invoke arbitrary code
Policy engine	Allow/deny/approval decision from tool and arguments	Decision is enforced before tool execution
Tool registry	Typed schemas and controlled tool adapters	Each tool validates input and limits output
Verification engine	Checks invariants, evidence and task success	Separates objective checks from model judging
Recovery manager	Retries transient failures or escalates	Bounded attempts, time and cost
Event recorder	Append ordered events and timing metadata	Redacts secrets; preserves run provenance
Evaluator	Scores runs and aggregates experiment metrics	Uses predeclared formulas and task ground truth
Persistence	Stores task definitions, runs, events, approvals and results	SQLite initially; optional PostgreSQL

4.3 Logical layers

- **Presentation:** Streamlit or static React UI; show run status, trace timeline, approvals, comparison and exports.
- **Application:** FastAPI endpoints and application services for tasks, runs, approvals and experiments.
- **Domain:** typed schemas, state transitions, policy decisions, event contracts and errors.
- **Agent runtime:** planner interface, executor, tool registry, verification and bounded recovery.
- **Data:** task definitions, benchmark assets, event store, run metadata and aggregated metrics.
- **Research:** experiment runner, ablation matrix, statistical analysis and report export.

4.4 Deployment view

Environment	Components	Purpose
Local development	Python package + SQLite + mock model	Fast tests and reproducible experiments
CI	Unit/integration/security tests + small benchmark	Catch regressions; avoid expensive model calls
Public demo	Free web API + static frontend + seeded synthetic data	Recruiter/reviewer walkthrough with strict limits
Research runs	Local or CI batch execution; artifacts exported	Repeatable experiments independent of hosting uptime

4.5 Main execution sequence

```
1. Client submits task_id + run configuration + limits.
2. API validates request and creates a run record.
3. Run manager loads task, dataset version and policy.
4. Orchestrator proposes a next action through the planner interface.
5. Tool registry validates tool name and argument schema.
6. Policy engine returns ALLOW, DENY or REQUIRE_APPROVAL.
7. If approved/allowed, executor runs the tool in the restricted environment.
8. Recorder stores tool request summary, outcome, timing and policy decision.
9. Verification engine checks evidence, invariants and task conditions.
10. Orchestrator completes, retries, replans, pauses or fails within hard limits.
11. Evaluator scores terminal state and writes metrics/report.
12. API returns run status, result summary and trace reference.
```

4.6 Key architecture decision records

Decision	Initial choice	Reason / revisit when
Workflow engine	Explicit state machine	Easy to inspect and test; adopt LangGraph if checkpointing complexity warrants it.
Default storage	SQLite	No paid database or setup; switch to PostgreSQL for concurrent hosted use.
Model	Deterministic mock + adapter interface	Guarantees offline tests; real models are optional experimental conditions.
Tool protocol	Internal typed interface, then MCP adapter	Keeps core simple while allowing standard tool interoperability.
UI	Streamlit or static frontend	Prioritize trace clarity over frontend complexity.
Vector database	Optional	Add Qdrant only for tasks where semantic retrieval is actually evaluated.

5. Low-Level Design (LLD)

5.1 Core domain objects

Object	Important fields	Purpose
TaskDefinition	task_id, title, goal, tool_allowlist, success_checks, dataset_version	Defines a benchmark or user task.
RunConfig	verification, risk_aware, recovery, max_steps, max_tool_calls, timeout, seed	Captures a run's experimental conditions.
Run	run_id, task_id, status, started_at, ended_at, config_hash, terminal_reason	Represents one execution.
AgentState	messages/observations, step_count, retries, current_plan, pending_approval_id	Transient state for the orchestrator.
ToolCall	tool_call_id, run_id, tool_name, arguments, status, result_ref, duration_ms	Auditable record of a tool invocation.
PolicyDecision	decision_id, action_hash, outcome, risk_level, rule_ids, reason	Records allow/deny/approval reasoning.
Approval	approval_id, run_id, action_hash, status, decided_by, decided_at	Binds human decision to an exact action.
VerificationResult	check_id, passed, severity, evidence_refs, message	Records outcome checks.
RunEvent	event_id, run_id, sequence, type, timestamp, payload_redacted	Ordered observability record.
Experiment	experiment_id, task_set_version, variant_matrix, repetitions, protocol_version	Defines a repeatable evaluation.
MetricRecord	run_id, metric_name, value, unit, formula_version	Stores raw per-run metric values.

5.2 Run state machine

Current state	Condition/event	Next state
CREATED	Validated request; limits and task loaded	READY
READY	Start execution	PLANNING
PLANNING	Planner returns final or tool action	EXECUTING / VERIFYING / COMPLETED
EXECUTING	Policy allows and tool returns	OBSERVING
EXECUTING	Policy denies	OBSERVING or FAILED_POLICY
EXECUTING	Approval required	WAITING_APPROVAL
WAITING_APPROVAL	Approval granted/rejected	EXECUTING / CANCELLED / FAILED_POLICY
OBSERVING	Tool result received	VERIFYING / PLANNING
VERIFYING	Checks pass	COMPLETED
VERIFYING	Recoverable issue within budget	RECOVERING
RECOVERING	Retry/replan within limits	PLANNING / EXECUTING
Any active state	Timeout, cancellation or hard limit	TIMED_OUT / CANCELLED / FAILED_LIMIT

Terminal states

COMPLETED, FAILED, FAILED_POLICY, FAILED_LIMIT, TIMED_OUT and CANCELLED are terminal. Keep 'task outcome' separate from 'execution status': an agent may execute safely but fail the task, or complete a task while recording an attempted policy violation.

5.3 Planner and executor contracts

```
class ToolSpec:
    name: str
    description: str
    input_schema: dict
    risk_level: RiskLevel
    requires_approval: bool
    timeout_seconds: int

class ToolResult:
    tool_call_id: UUID
    status: ToolStatus
    output: dict | None
    error_code: str | None
    duration_ms: int
    evidence_refs: list[str]

class PolicyDecision:
    outcome: Literal["allow", "deny", "require_approval"]
    risk_level: RiskLevel
    matched_rules: list[str]
    action_hash: str
    reason: str

class VerificationResult:
    passed: bool
    checks: list[CheckResult]
    failure_class: str | None
    evidence_refs: list[str]
```

5.4 Policy decision flow

```
validate tool name and typed arguments
if tool not in task_allowlist: DENY
if arguments violate schema or hard limits: DENY
compute action hash from canonical tool + arguments + run context
evaluate deterministic rules and risk classification
if policy says prohibited: DENY
if approval is required and no matching approval exists: PAUSE
if approval exists but action hash differs: require fresh approval
otherwise: ALLOW
execute only through the registered tool adapter
record decision and outcome before returning to the agent
```

5.5 Verification flow

- Validate tool output against its declared schema.
- Run deterministic assertions for known task conditions (e.g., totals, row counts, required fields).
- Check that cited evidence references exist and correspond to returned tool outputs.
- Check invariants, such as read-only tools not mutating the environment.
- If checks fail, classify the issue as recoverable, non-recoverable, or requiring human review.
- Do not equate a model's statement that it is done with verified task completion.

5.6 Recovery algorithm

```
on recoverable failure:
    if retry_count >= max_retries_per_step: terminate or escalate
    if global step/tool/time/cost budget is exhausted: stop
    if error is transient and retry is idempotent: bounded retry
    else if replanning is enabled: return a sanitized failure observation
    else: stop with explicit failure
record every attempt and the final recovery outcome
```

6. Data Model and Persistence

6.1 Logical schema

Table	Primary key	Selected columns / relationships
tasks	task_id UUID	title, goal, task_json, dataset_version, created_at
runs	run_id UUID	task_id FK, status, config_json, config_hash, started_at, ended_at, terminal_reason
run_events	event_id UUID	run_id FK, sequence_no, event_type, timestamp, payload_json, payload_hash
tool_calls	tool_call_id UUID	run_id FK, tool_name, argument_hash, status, duration_ms, result_ref
policy_decisions	decision_id UUID	run_id FK, action_hash, outcome, risk_level, rule_ids, reason
approvals	approval_id UUID	run_id FK, action_hash, status, reviewer, decided_at, decision_note
verification_checks	check_id UUID	run_id FK, check_name, passed, severity, evidence_refs_json
experiments	experiment_id UUID	name, protocol_version, task_set_hash, matrix_json, repetitions
metric_records	metric_id UUID	experiment_id FK, run_id FK, metric_name, value, unit, formula_version

6.2 Persistence rules

- Use UUIDs for externally visible identifiers; use monotonically increasing sequence numbers to order events within a run.
- Persist the exact configuration and benchmark version for every run.
- Canonicalize tool arguments before hashing; do not store secrets in the hash input logs or event payloads.
- Use transactions for run-state changes and approval decisions.
- Make event recording append-oriented; corrections should create new events rather than silently rewrite history.
- Store large outputs as artifacts and keep only a reference, checksum and safe summary in the event record.
- Use SQLite in the local prototype; design repository interfaces so PostgreSQL can be added without rewriting domain logic.

6.3 Benchmark task format

```
{
  "task_id": "analytics_001",
  "domain": "analytics",
  "goal": "Calculate failed-order rate for the supplied date range.",
  "tool_allowlist": ["readonly_sql", "calculator"],
  "success_checks": [
    {"type": "numeric_match", "target": "failed_order_rate",
     "tolerance": 0.0001}
  ],
  "risk_policy": {"write_operations": "deny"},
  "fault_scenarios": ["temporary_tool_timeout"],
  "dataset_version": "orders-demo-v1"
}
```

6.4 Reproducibility manifest

- Project commit hash and dependency lockfile hash.
- Benchmark version and dataset checksum.
- Experiment protocol version and variant configuration.
- Model provider/name/version and decoding parameters where available.
- Seed, execution environment, limits and fault scenario.
- Raw per-run results, excluded-run reasons, and report-generation version.

7. API Specification

Base path: /api/v1. FastAPI should publish OpenAPI documentation. Use UUIDs for task/run/approval/experiment IDs and a consistent error envelope.

Method	Endpoint	Purpose
GET	/health	Liveness/version status
POST	/tasks	Create task definition
GET	/tasks/{task_id}	Read task definition
POST	/runs	Start a bounded run
GET	/runs/{run_id}	Get status/result
POST	/runs/{run_id}/cancel	Request cancellation
GET	/runs/{run_id}/events	Paginated ordered events
GET	/runs/{run_id}/trace	Trace summary and tool sequence
GET	/runs/{run_id}/approvals	List approvals
POST	/approvals/{approval_id}/decision	Approve/reject exact action
POST	/experiments	Create experiment definition
POST	/experiments/{experiment_id}/execute	Start bounded experiment
GET	/experiments/{experiment_id}/results	Metrics and comparisons
GET	/experiments/{experiment_id}/report	Export report artifact

7.1 Example run request

```
{
  "task_id": "7c58d0e4-8ed0-4b1d-9a0a-8b85e3f4e111",
  "configuration": {
    "verification": true,
    "risk_aware_policy": true,
    "recovery_enabled": true
  },
  "limits": {
    "max_steps": 12,
    "max_tool_calls": 20,
    "max_retries_per_step": 2,
    "timeout_seconds": 45
  },
  "seed": 42
}
```

7.2 Example response

```
{
  "run_id": "<server-generated-uuid>",
  "status": "completed",
  "task_outcome": "passed",
  "terminal_reason": "all_success_checks_passed",
  "metrics": {
    "tool_calls": 4,
    "duration_ms": 2180,
    "policy_violations": 0
  },
  "trace_url": "/api/v1/runs/<run-id>/trace"
}
```

The numbers above are illustrative only. Do not publish them as project results.

7.3 Approval request

```
POST /api/v1/approvals/{approval_id}/decision
{
  "decision": "approve",
  "action_hash": "<hash shown to reviewer>",
  "note": "Approved for the synthetic demo environment."
}
```

7.4 Error envelope

```
{
  "error": {
    "code": "RUN_LIMIT_EXCEEDED",
    "message": "The run reached its configured tool-call limit.",
    "request_id": "<uuid>",
    "details": {}
  }
}
```

7.5 API design rules

- Validate every request and tool result using explicit Pydantic models.
- Enforce upper bounds on every user-supplied limit.
- Return 400 for invalid input, 401/403 for auth/permission failures, 404 for unknown resources, 409 for invalid state transitions, and 429 for rate limits.
- Use idempotency keys for run creation if clients may retry requests.
- Use pagination for events and traces; redact sensitive fields before persistence and response.
- For the public demo, expose only seeded tasks or require a demo token for run creation and approval decisions.

8. Security, Safety and Reliability

AgentOps evaluates agents; it must not become an unrestricted agent execution service. Security controls belong in the runtime and tool adapters, not only in prompts.

8.1 Threat model

Threat	Mitigation
Prompt injection in task/data	Treat retrieved content as untrusted data; tool permissions are independent of model instructions.
Unauthorized tool call	Server-side allowlist, schema validation and policy check before execution.
Approval replay or tampering	Bind approval to canonical action hash, run, expiry and reviewer; invalidate if action changes.
SQL abuse	Read-only connection, query restrictions, statement timeout, row limit and synthetic database.
Code execution	No arbitrary shell or Python execution in public demo; isolate any future execution in a hardened sandbox.
Resource exhaustion	Rate limits, step/call/time/output caps, bounded concurrency and request size limits.
Secret leakage	Environment secrets, log redaction, no secrets in trace payloads, dependency and repository scans.
Trace tampering	Append-oriented events, sequence checks, optional payload hashes and immutable research artifacts.
Public endpoint abuse	Authentication or demo token, quotas, restrictive CORS and no destructive real-world tools.

8.2 Reliability controls

- Timeout every tool invocation and model request.
- Retry only transient errors and respect idempotency.
- Cap retries, total steps, tool calls, wall-clock time and output size.
- Handle cancellation and terminal states explicitly.
- Record exceptions with stable error codes and sanitized messages.
- Separate system health from task success: HTTP 200 does not mean the agent completed the task correctly.
- Use deterministic policy and verification tests before running real-model experiments.

8.3 Privacy and data handling

Use synthetic or licensed redistributable datasets. Do not upload private documents or production credentials into the public demo. Establish a documented retention policy for runs and artifacts, and allow exported traces to be scrubbed before publication.

9. Evaluation Protocol

9.1 Metrics and formulas

Metric	Definition / interpretation
Task success rate	Runs passing all predefined success checks / eligible runs.
Undetected failure rate	Failed tasks incorrectly accepted as successful / all tasks that truly failed.
Policy violation rate	Prohibited actions actually executed / attempted tasks; also report attempted violations separately.
Recovery success rate	Recoverable failures followed by successful completion / recoverable failures encountered.
Tool-call efficiency	Tool calls per successful task; report failures and timeouts separately.
Latency	End-to-end wall-clock time; report median and p95.
Execution cost	Measured model/API cost plus any attributable compute cost; report unknown costs explicitly.
Approval correctness	Correct approval/denial decisions against predeclared policy / evaluated decisions.
False-denial rate	Allowed actions incorrectly denied / actions that should be allowed.

9.2 Benchmark domains

- **Analytics:** calculate metrics from small synthetic tables, verify arithmetic and query results.
- **Document investigation:** answer questions from versioned sample documents with exact source references.
- **Operations:** propose simulated changes and enforce approval before a state transition.
- **Fault injection:** timeout, malformed response, missing evidence, conflicting results, denied tool, transient failure.

9.3 Experimental matrix

Variant	Verification	Risk-aware policy	Recovery
Baseline	Off	Off / uniform	Off
V	On	Off / uniform	Off
P	Off	On	Off
R	Off	Off / uniform	On
Full	On	On	On

This is an initial matrix. A complete 2×2×2 factorial experiment has eight combinations and can reveal interactions. Choose the final design before collecting results.

9.4 Statistical analysis

- Use matched task IDs and initial states across variants.
- Report sample size, missing runs, exclusions and reasons.
- Provide confidence intervals and effect sizes, not only point estimates.
- For paired binary outcomes, consider McNemar's test where assumptions fit; for paired continuous outcomes, select a suitable paired test or bootstrap.
- Account for repeated runs nested within tasks; do not treat correlated runs as independent observations.
- Predefine primary outcomes to limit selective reporting and clearly label exploratory analyses.
- Use an independent test split if tasks or prompts are tuned during development.

9.5 Minimum dataset plan

Start with 30–50 hand-authored tasks to validate the instrumentation and task format. This is an engineering milestone, not a claim that the sample size is statistically sufficient. Expand after a pilot, estimate variability, and justify the final sample size for the primary comparison.

9.6 Results reporting template

Variant	N tasks	Success %	Undetected failures %	Policy violations	Median / p95 latency	Cost / task
Baseline	To measure	To measure	To measure	To measure	To measure	To measure
Verification	To measure	To measure	To measure	To measure	To measure	To measure
Risk-aware	To measure	To measure	To measure	To measure	To measure	To measure
Recovery	To measure	To measure	To measure	To measure	To measure	To measure
Full system	To measure	To measure	To measure	To measure	To measure	To measure

10. Repository Structure and Build Plan

10.1 Target repository tree

```
agentops-intelligence/
■■■ README.md  LICENSE  CITATION.cff  pyproject.toml  uv.lock
■■■ Makefile  .env.example  Dockerfile  compose.yaml
■■■ docs/
■   ■■■ architecture.md  threat-model.md  research-protocol.md
■   ■■■ api.md  benchmark-card.md  deployment.md
■■■ configs/
■   ■■■ default.yaml  policies.yaml  models.yaml
■   ■■■ experiments/{baseline,verification,risk_aware,full_system}.yaml
■■■ src/agentops/
■   ■■■ api/{main.py,dependencies.py,errors.py,routes/}
■   ■■■ domain/{schemas.py,enums.py,events.py,exceptions.py}
■   ■■■ orchestrator/{engine.py,state.py,planner.py,executor.py,recovery.py,limits.py}
■   ■■■ policies/{engine.py,rules.py,risk.py,approvals.py}
■   ■■■ tools/{base.py,registry.py,calculator.py,readonly_sql.py,
■   ■          dataset_search.py,simulated_actions.py,fault_injection.py,mcp_adapter.py}
■   ■■■ verification/{engine.py,evidence.py,assertions.py,invariants.py,outcome_checks.py}
■   ■■■ observability/{recorder.py,events.py,tracing.py,redaction.py}
■   ■■■ evaluation/{runner.py,metrics.py,scorer.py,aggregation.py,statistics.py,reports.py}
■   ■■■ datasets/{loader.py,manifest.py,validation.py}
■   ■■■ persistence/{models.py,repositories.py,sqlite.py,postgres.py}
■■■ benchmark/{task_schema.json,tasks/,fault_scenarios/,ground_truth/,splits/}
■■■ frontend/{package.json,src/}
■■■ experiments/{run_matrix.py,ablation.py,analyze_results.py,notebooks/}
■■■ scripts/{seed_demo.py,validate_benchmark.py,export_results.py}
■■■ tests/{unit/,integration/,security/,benchmark/,fixtures/}
```

10.2 Six-week implementation plan

Week	Deliverable	Exit criterion
1	Deterministic execution core	Typed tools, bounded state machine, event recording and unit tests pass.
2	Policies and verification	Allow/deny/approval, assertions, fault injection and bounded recovery work.
3	Benchmark dataset	Task schema, initial tasks, ground truth and benchmark validation script.
4	Evaluation framework	Baseline and ablations run; metrics and raw traces export correctly.
5	API and UI	Task/run/trace/approval endpoints and a restricted demo interface.
6	Reproducibility and paper draft	Repeatable commands, result analysis, documentation and paper skeleton.

10.3 Testing strategy

- Unit tests: schema validation, policy rules, action hashing, state transitions, metric formulas and redaction.
- Integration tests: end-to-end run, approval pause/resume, tool timeout, malformed tool result and limit exhaustion.
- Security tests: disallowed tool, argument injection, SQL restrictions, approval mismatch and oversized payloads.
- Benchmark tests: ground-truth consistency, required metadata, valid fault references and no duplicate task IDs.
- Regression tests: keep a small fixed suite in CI; run the full experiment matrix manually or in a scheduled job.

11. ■0 Deployment and Operations Plan

11.1 Recommended topology

Part	Initial choice	Limitations and fallback
Source + CI	Public GitHub repository and automated tests	Keep secrets out of repository; CI minutes and quotas may apply.
API	Free web service such as Render, if current account terms permit	Free instances may sleep, have ephemeral disk, limited resources and request/runtime constraints.
UI / docs	GitHub Pages or another free static host	Static hosting cannot run the agent backend by itself.
Persistence	SQLite for local/short-lived demo	Do not assume local disk survives restarts on ephemeral hosting; reseed synthetic data.
LLM	Deterministic mock by default	Real model inference requires a currently available permitted free endpoint or local run.
Research experiments	Local/CI batch scripts and exported artifacts	Do not depend on the public API being awake or on long-running web requests.

11.2 Deployment steps

- Make local install and tests work from a clean environment using the lockfile.
- Implement /api/v1/health and a seeded sample task.
- Build a minimal Docker image or configure the host's Python build/start commands.
- Deploy the API with the platform-provided port and bind to 0.0.0.0.
- Configure demo authentication, rate limits, restrictive CORS and hard run limits.
- Deploy static frontend/docs separately; configure the public API base URL.
- Test after idle sleep/restart, missing model credentials, timeouts, invalid requests and rate limits.
- Export experiment results as versioned JSON/CSV/Markdown artifacts; keep the paper reproducible offline.

11.3 Free-tier caveats

Free tiers change and may have usage limits, sleeping services, ephemeral filesystems, restricted compute, and no guaranteed availability. Verify current provider terms at deployment time. Avoid making the public demo depend on a free database whose data expires or on a model endpoint with uncertain availability.

11.4 Demo safety checklist

- Use synthetic data and non-destructive simulated actions.
- No arbitrary code execution or unrestricted SQL.
- Set strict request size, step, call, timeout and concurrency limits.
- Require approval for any simulated high-risk state change.
- Redact secrets from traces and errors.
- Add a reset/reseed path and graceful handling for service sleep or restart.

12. Publication, Demo and Career Positioning

12.1 Paper outline

Section	Content
Abstract	Problem, proposed method, benchmark, experimental setup and measured results—written after results exist.
Introduction	Why final-answer quality is insufficient; research question and contributions.
Related work	Agent benchmarks, tool safety, runtime verification, recovery, observability.
Method	Policy engine, verification checks, recovery algorithm and execution limits.
Benchmark	Tasks, synthetic environments, fault scenarios, ground truth and splits.
Experiments	Baselines, ablations, models, repetitions, metrics and statistical methods.
Results	Measured outcomes, confidence intervals, costs, failure examples and trade-offs.
Discussion	Interpretation, limitations, threats to validity and external generalizability.
Reproducibility	Code, environment, configs, task manifest, artifacts and run instructions.

12.2 Artifacts to publish

- Public source repository with license and citation metadata.
- Versioned benchmark tasks, schemas, synthetic data and ground truth.
- Configuration files and scripts for baselines and ablations.
- Raw sanitized traces and aggregate result files.
- Benchmark card describing scope, intended use, known limitations and task-generation process.
- Threat model and documented security assumptions.
- Short screen-recorded demo and architecture diagram.
- Paper preprint only after results and related-work review are complete.

12.3 Portfolio description (use only after implementation)

AgentOps Intelligence — Reliable Tool-Using AI Agents
Designed a research-oriented platform to evaluate tool-using agents under controlled failures and risk constraints. Implemented bounded tool execution, policy enforcement, evidence-based verification, human approval workflows and structured execution traces. Built a reproducible benchmark to measure task success, failure detection, recovery, policy violations, latency and execution cost.

Do not claim performance improvements, publication acceptance, production use, or features that have not been implemented and measured.

12.4 Resume bullet template

Built [implemented modules] using [actual stack]; evaluated [number] benchmark tasks across [number] configurations and measured [actual metric changes], publishing reproducible scripts and sanitized traces. Replace every bracketed field with verified project data.

12.5 Suggested 90-second demo

- Show one benchmark task and its declared success criteria.
- Run the baseline and inspect the actual tool trace and failure outcome.
- Run the verification/recovery variant on the same task and initial state.
- Show the policy decision, evidence checks and terminal status.

- Compare success, calls, latency and cost in the dashboard.
- Open the benchmark config and show the command that reproduces the run.

13. Reference Reading and Final Checklist

The following sources are starting points for architecture and literature review. They do not establish the novelty or effectiveness of the proposed method.

Topic	Starting point
Agent evaluation research	https://arxiv.org/ (search recent surveys on LLM-agent evaluation and benchmarks)
LangGraph persistence and human-in-the-loop	https://docs.langchain.com/oss/python/langgraph/persistence
Model Context Protocol tools	https://modelcontextprotocol.io/specification/2025-06-18/server/tools
FastAPI documentation	https://fastapi.tiangolo.com/
Pydantic documentation	https://docs.pydantic.dev/
Render free services	https://render.com/docs/free
GitHub Actions documentation	https://docs.github.com/actions

Final readiness checklist

- Core execution is bounded and every tool call is policy-checked.
- Approval decisions bind to an exact action and are recorded.
- The benchmark includes ground truth and controlled failure scenarios.
- Baselines, ablations and metrics are defined before final evaluation.
- Statistical uncertainty and negative results are reported.
- The public demo uses synthetic data, restricted tools and safe limits.
- The repository reproduces the main results from documented commands.
- The paper's novelty claims are supported by a completed literature review.

Project principle

Build the smallest system that can prove or disprove the research hypotheses. A clear benchmark, reliable instrumentation and honest analysis are more valuable than a complex agent that cannot be evaluated.

End of blueprint • Version 1.0